🐬 MySQL

Меняем слоника на дельфинчика

Некрасов Ярослав Алексеевич

👴 Историческая справка

1986 год

В Америке 🇺🇸🦅

В университете Беркли придумали PostgreSQL

Он требовал полного понимания происходящего

Из принципов не продаются какой-то компании и оставляют код открытым

Он дает много возможностей для настройки и расширения

Чтобы визуально с ним работать мы использовали pgAdmin

Это база

1995 год

В Швеции 🇸🇪

Делали с расчетом на простоту и скорость

Легче подключать к веб-приложениям

Продался компании Oracle
(разработчики Java)

Небольшие команды выполняются быстрее

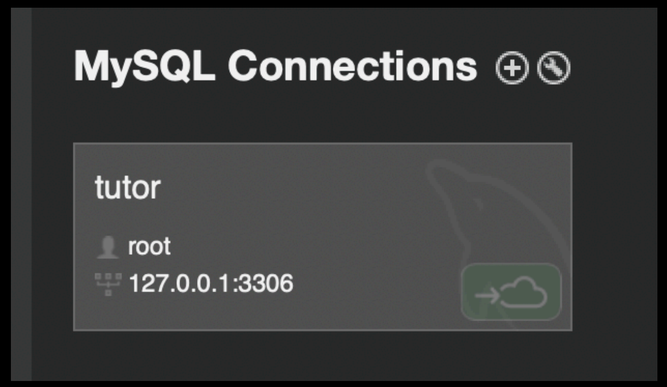
Не полностью соответствует всем
стандартам SQL

Мы начали с PostgreSQL чтобы было понимание строгого SQL, дальше если вы будете изучать другие СУБД - будет больше сходств

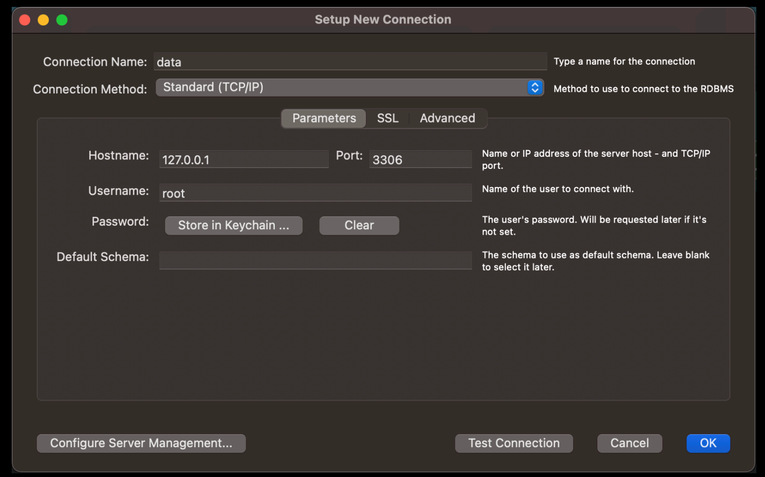
База MySQL

Создаем базу данных

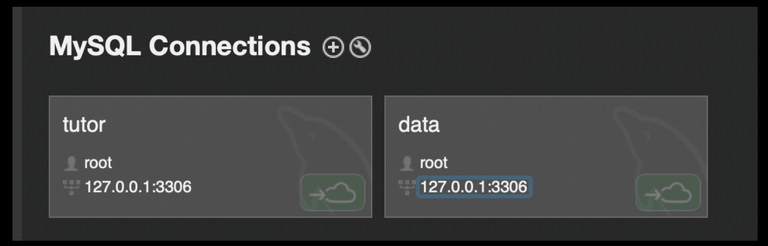
Жмем на плюс



Вводим connection name



Нажимаем 2 раза на пользователя

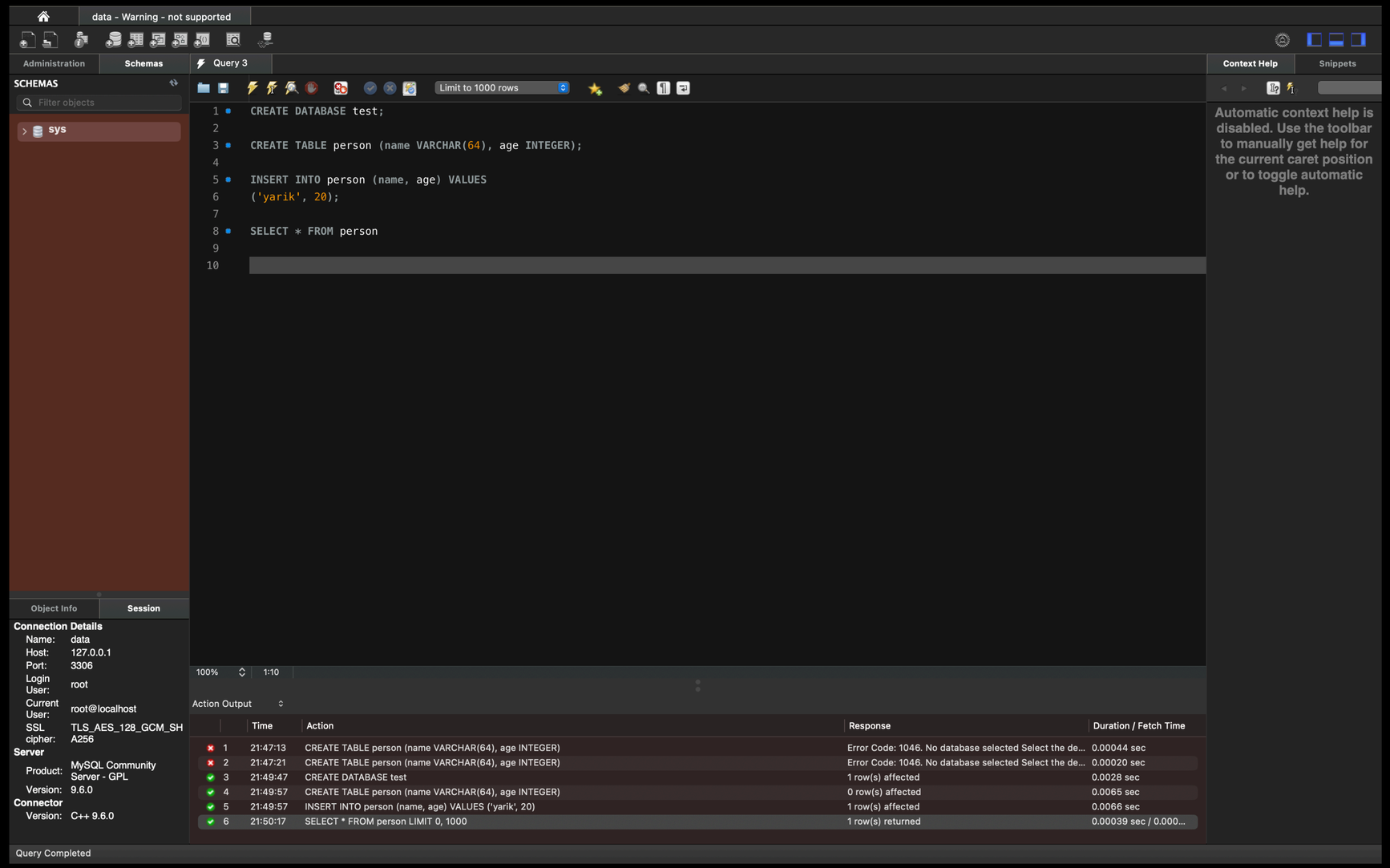


MySQL

Интерфейс

Мы можем делать запросы с главной страницы

Чтобы его запустить - нужно нажать на молнию ⚡



MySQL

Создаем базу данных

Создаем базу

sql - script.sql

```sql
CREATE DATABASE baza;
```

В запросе указываем какую базу использовать

sql - script.sql

```sql
USE baza;

CREATE TABLE person (name VARCHAR(64), age INTEGER);

INSERT INTO person (name, age) VALUES
('yarik', 20);

SELECT * FROM person
```

PostgreSQL и MySQL

Смотрим разницу

🐘 **PostgreSQL**

sql - script.sql

```sql
-- Адрес
CREATE TABLE address (
    id SERIAL PRIMARY KEY,
    city VARCHAR(64),
    street VARCHAR(64),
    house VARCHAR(32)
);

INSERT INTO address (city, street, house) VALUES
('Новосибирск', 'Красный проспект', 1),
('Новосибирск', 'Красный проспект', 2),
('Новосибирск', 'Красный проспек', 3);


-- Люди
CREATE TABLE person (
    id SERIAL PRIMARY KEY,
    name VARCHAR(64),
    age INTEGER,
    address_id INTEGER,
    FOREIGN KEY (address_id) REFERENCES address(id)
);

INSERT INTO person (name, age, address_id) VALUES
('ярик', 20, 1),
('леша', 21, 2),
('данил', 19, 3);

SELECT p.name, p.age, a.city, a.street, a.house
FROM person p
JOIN address a ON a.id = p.address_id;

```

🐬 **MySQL**

sql - script.sql

```sql
-- Адрес
CREATE TABLE address (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, --  <-- Другой тип
    city VARCHAR(64),
    street VARCHAR(64),
    house VARCHAR(32)
);

INSERT INTO address (city, street, house) VALUES
('Новосибирск', 'Красный проспект', 1),
('Новосибирск', 'Красный проспект', 2),
('Новосибирск', 'Красный проспек', 3);

-- Люди
CREATE TABLE person (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(64),
    age INTEGER,
    address_id INT UNSIGNED, --  <--  Другой тип
    FOREIGN KEY (address_id) REFERENCES address(id)
);


INSERT INTO person (name, age, address_id) VALUES
('ярик', 20, 1),
('леша', 21, 2),
('данил', 19, 3);

SELECT p.name, p.age, a.city, a.street, a.house
FROM person AS p --  <-- Слово появилось
JOIN address AS a ON a.id = p.address_id; --  <-- Слово появилось
```

PostgreSQL и MySQL

Смотрим разницу

UNSIGNED - беззнаковый (не может быть отрицательным)

AUTO_INCREMENT - автоматическое увеличение (в PostreSQL - **SERIAL**)

🐬 MySQL

sql - script.sql

```sql
-- Адрес
CREATE TABLE address (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, --  <-- Другой тип
    city VARCHAR(64),
    street VARCHAR(64),
    house VARCHAR(32)
);

INSERT INTO address (city, street, house) VALUES
('Новосибирск', 'Красный проспект', 1),
('Новосибирск', 'Красный проспект', 2),
('Новосибирск', 'Красный проспек', 3);

-- Люди
CREATE TABLE person (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(64),
    age INTEGER,
    address_id INT UNSIGNED, --  <--  Другой тип
    FOREIGN KEY (address_id) REFERENCES address(id)
);


INSERT INTO person (name, age, address_id) VALUES
('ярик', 20, 1),
('леша', 21, 2),
('данил', 19, 3);

SELECT p.name, p.age, a.city, a.street, a.house
FROM person AS p --  <-- Слово появилось
JOIN address AS a ON a.id = p.address_id; --  <-- Слово появилось
```

PostgreSQL и MySQL

Смотрим разницу в функциях

🐘 PostgreSQL

sql - script.sql

```sql
CREATE FUNCTION get_person(key INTEGER)
RETURNS TABLE(
    name VARCHAR(64),
    age INTEGER,
    city VARCHAR(64),
    street VARCHAR(64),
    house VARCHAR(32))
LANGUAGE plpgsql
AS $$
BEGIN
RETURN QUERY
    SELECT p.name, p.age, a.city, a.street, a.house
    FROM person p
    JOIN address a ON p.address_id = a.id
    WHERE p.id = key;
END;
$$;

SELECT * FROM get_person(1)
```

🐬 MySQL

sql - script.sql

```sql
DELIMITER //
CREATE PROCEDURE get_person(IN p_key INT)
BEGIN
    SELECT
        p.name,
        p.age,
        a.city,
        a.street,
        a.house
    FROM person p
    JOIN address a ON p.address_id = a.id
    WHERE p.id = p_key;
END //
DELIMITER ;

CALL get_person(1);
```

PostgreSQL и MySQL

Смотрим разницу в функциях

В MySQL процедуры более гибкие и ими спокойно заменяют функции

DELIMITER нужен чтобы обозначить начало и конец функции

🐬 MySQL

sql - script.sql

```sql
DELIMITER //
CREATE PROCEDURE get_person(IN p_key INT)
BEGIN
    SELECT
        p.name,
        p.age,
        a.city,
        a.street,
        a.house
    FROM person p
    JOIN address a ON p.address_id = a.id
    WHERE p.id = p_key;
END //
DELIMITER ;

CALL get_person(1);
```

Задание

Реализовать базу данных из КТ 3 на MySQL

Отправить pdf / docx с:

- Выводом процедур и SELECT
- SQL запросом